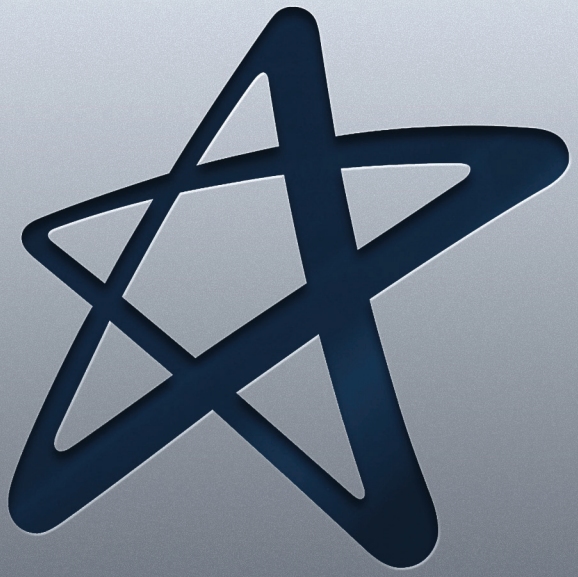
Linguagens Formais e Autômatos



Cruzeiro do Sul Virtual
Educação a distância

Material Teórico

Conceitos Básicos

Responsável pelo Conteúdo:
Prof. Dr. Cléber Silva Ferreira da Luz

Revisão Textual:
Prof. Esp. Claudio Pereira do Nascimento

UNIDADE Conceitos Básicos



- Introdução;
- Conceitos Básicos de Conjuntos;
- Conceitos Fundamentais de Linguagem Formal.
- Conclusões e Resumo da Unidade.

OBJETIVO DE APRENDIZADO

- Apresentar os conceitos básicos sobre linguagens formais.

Orientações de estudo

Para que o conteúdo desta Disciplina seja bem aproveitado e haja maior aplicabilidade na sua formação acadêmica e atuação profissional, siga algumas recomendações básicas:



Assim:

- ✓ Organize seus estudos de maneira que passem a fazer parte da sua rotina. Por exemplo, você poderá determinar um dia e horário fixos como seu "momento do estudo";
- ✓ Procure se alimentar e se hidratar quando for estudar; lembre-se de que uma alimentação saudável pode proporcionar melhor aproveitamento do estudo;
- ✓ No material de cada Unidade, há leituras indicadas e, entre elas, artigos científicos, livros, vídeos e *sites* para aprofundar os conhecimentos adquiridos ao longo da Unidade. Além disso, você também encontrará sugestões de conteúdo extra no item **Material Complementar**, que ampliarão sua interpretação e auxiliarão no pleno entendimento dos temas abordados;
- ✓ Após o contato com o conteúdo proposto, participe dos debates mediados em fóruns de discussão, pois irão auxiliar a verificar o quanto você absorveu de conhecimento, além de propiciar o contato com seus colegas e tutores, o que se apresenta como rico espaço de troca de ideias e de aprendizagem.

Introdução

Originalmente proposta em 1950, a teoria da linguagem formal visa o estudo das teorias relacionadas com as linguagens naturais. A teoria da linguagem formal estuda modelos matemáticos que visam à especificação e o reconhecimento de linguagens naturais, tais como suas classificações, estruturas e propriedades. Ao longo dos anos, foi constatado que a teoria das linguagens formais também era importante para o estudo das linguagens artificias, como as originárias da computação (MENEZES, P. B,2011).

A linguagem formal pode ser aplicada em:

- modelagem de circuitos lógicos ou redes lógicas;
- sistemas biológicos;
- sistemas de animações;
- hipertextos e hipermídias;
- reconhecimento de padrões; e
- principalmente nas linguagens de programação.

Nas linguagens de programação, a teoria das linguagens formais possui papel fundamental nas análises léxica e sintática das linguagens de programação (MENEZES, P. B, 2011).

Conceitos Básicos de Conjuntos

Na literatura, é possível encontrar a definição de linguagem como "**um conjunto de símbolos**" (RAMOS, Marcus Vinicios Midena; VEGA, Italo Santiago; JOSE NETO, Joao, 2009). Nesta seção, estudaremos as principais definições e conceitos sobre conjuntos, tais definições e conceitos são fundamentais para o estudo sobre a teoria das linguagens formais e autômatos.

Conjunto

Um **conjunto** pode ser visto como uma **coleção** de **símbolos** onde não são considerados ocorrências múltiplas destes, como, também, não há relação de ordem entre eles. Símbolos também são chamados de átomos ou **elementos**, (RAMOS, Marcus Vinicios Midena; VEGA, Italo Santiago; JOSE NETO, Joao, 2009).

Conjuntos podem ser **especificados** através da simples **enumeração** de **todos** os seus **elementos**, **denotados** entre **chaves** e **separados** por **vírgulas** (RAMOS, Marcus Vinicios Midena; VEGA, Italo Santiago; JOSE NETO, Joao, 2009). Por exemplo, o conjunto formado pelos símbolos ♥ ♦ ♣ ♠ pode ser representado

por {♥, ♦, ♣, ♠}. Assim, como o conjunto formado pelas letras **a b c d e** pode ser representado por {a, b, c, d, e}. **Conjuntos** também podem ser **referenciados** por **nomes** arbitrariamente escolhidos. Exemplos: **A = {1,2,3,4,5}**; **B = {a,b,c,d,e,f,g}.** Dessa forma, os nomes A e B passam a referenciar os conjuntos correspondentes (RAMOS, Marcus Vinicios Midena; VEGA, Italo Santiago; JOSE NETO, Joao, 2009). É importante observar que sempre o nome do conjunto será denotado por símbolos maiúsculos e seus elementos por símbolos minúsculos.

A **quantidade** de **elementos contido** em um **conjunto** é **denotada** por | |.

Por exemplo, considerando os conjuntos A e B exemplificado anteriormente, temos; |A| = 5, |B| = 7.

Os símbolos $\in$ e $\notin$ são utilizados para denotar se um elemento **pertence** ou **não pertence** a um conjunto, respectivamente (RAMOS, Marcus Vinicios Midena; VEGA, Italo Santiago; JOSE NETO, Joao, 2009). Por exemplo, 2 $\in$ A, 5 $\in$ A, 9 $\notin$ A, a $\in$ B, d $\in$ B e h $\notin$ B.

Um **conjunto** pode conter um número **finito** ou **infinito** de elementos (RAMOS, Marcus Vinicios Midena; VEGA, Italo Santiago; JOSE NETO, Joao, 2009). Por exemplo, o conjunto A e o conjunto B são exemplos de conjuntos finitos porque ambos possuem um número finito de elementos.

Um **conjunto infinito** é caracterizado por possuir um número infinito de elementos (MENEZES, P. B,2011). Por exemplo, o conjunto dos números pares é um conjunto infinito, pois contém um número infinito de números. Habitualmente, conjuntos infinitos são denotados através da especificação de regras ou propriedades que devem ser satisfeitas por todos os seus elementos, possibilitando, assim, a sua identificação precisa e completa a partir de uma especificação finita.

Exemplos:

> $P = \{ x \mid x$ é um número par$\}$. Este exemplo deve ser lido da seguinte maneira: "P é o conjunto formado pelos elementos *x*, tal que *x* é um número par, isto é, P é o conjunto infinito formado por todos os números pares".

É importante observar o símbolo " | ". Quando um conjunto é especificado e utiliza regras, o símbolo " | " deve ser lido como "tal que" e é utilizado para introduzir as condições que devem ser satisfeitas pelos elementos do conjuntos que, assim, se tornam conhecidos.

Um **conjunto** que **não contém nenhum elemento** é denominado como **conjunto vazio**. O conjunto vazio é **representado** por ∅. Um conjunto vazio também pode ser representado por { }. Dessa forma, { } = ∅. Por definição, |∅| = 0.

Conjuntos podem ser **idênticos**. Dois conjuntos são ditos idênticos, isto é, iguais, se eles contêm exatamente os mesmos elementos. A **igualdade** entre dois conjuntos é **denotada** pelo **símbolo " = "**. Exemplo, considere $X = \{a, b\}$ e $Y = \{b, a\}$. Então, $X = Y$.

Um conjunto pode estar **contido** em outro conjunto. Um conjunto Y está contido no conjunto Z caso todos os elementos de Y estejam em Z, denotado, assim, por; $Y \subseteq Z$. Por exemplo, considere os conjuntos **X = {2,5,6}**, **Y = {1,2,3,4,5,6,7,8}** e **Z = {8,5,1,7,3,4,2,6}**. Assim, $Y \subseteq Z$. Por outro lado, Y não está contido em X, assim como Z não está contido em X.

Quando, por exemplo, $Y \subseteq Z$, podemos falar que **Y** é um **subconjunto** de **Z**, ou, ainda, que **Z contém Y**. Por definição, ∅ **é um subconjunto de qualquer conjunto**. Por exemplo, ∅ é um **subconjunto de X** (MENEZES, P. B,2011).

Dois conjuntos podem ser **iguais se** e **somente** $Y \subseteq Z$ e $Z \subseteq Y$. Essa igualdade é representada pelo símbolo "=", assim, **Y = Z**. Já a **desigualdade** entre dois conjuntos é representada pelo **símbolo** ≠ e é dada quando no máximo apenas uma das condições $Y \subseteq Z$ e $Z \subseteq Y$ for verdadeira. Assim, $X \subseteq Y$, todavia, $X \neq Y$ e $X \neq Z$. O símbolo "≠" representa a desigualdade de dois conjuntos.

Se $X \subseteq Y$ e $X \neq Y$, diz que **X** é um **subconjunto próprio** de **Y**. Para representar essa condição, é utilizado o **símbolo** $\subset$. Assim, $X \subset Y$. Por definição, o ∅ é **subconjunto próprio** de qualquer conjunto, menos do próprio conjunto ∅. Portanto, X é um subconjunto de Y e $X \subset Y$.

Operações sobre conjunto

Em determinados momentos, pode ser necessário realizar algumas operações sobre os conjuntos como, por exemplo, a união entre dois conjuntos. Agora, iremos estudar as principais operações sobre conjuntos.

Operação de potência

A operação de **potência** é denotada por $\mathbf{2^A}$, onde **A** é um **conjunto**. A principal utilidade desta operação é designar o **conjunto formado** por **todos os possíveis subconjuntos de A**:

$$2^A = \{B \mid B \subseteq A\}$$

Para conjuntos **A** finito, o seu tamanho é definido por $|2^A| = 2^{|A|}$. Por exemplo, considere os conjuntos A = {0, 1, 2}, temos 2^A = { ∅, {0}, {1}, {2}, {0,1}, {02}, {1,2}, {0,1,2}}. Também, $|A| = 3$ e $|2^A| = 2^3 = 8$.

Operação de união

A operação de **união** entre **dois conjuntos A** e **B** corresponde ao **conjunto formado** por **todos os elementos contidos em cada um dos conjuntos A e B**. A operação de união é representada pelo símbolo ∪. A operação de união de A e B é representada por $A \cup B$. Caso haja elementos repetidos em ambos os conjuntos, são considerados uma única vez no conjunto formado pela união dos conjuntos:

$$\boldsymbol{A \cup B = \{x \mid x \in A \text{ ou } A \in B\}}$$

A união é uma operação associativa, isto é, uma operação a qual vale a propriedade:

$$(A \cup B) \cup C = A \cup (B \cup C)$$

Exemplos:

- $\{1,2,3,4,\} \cup \{0,1,2,5,6\} = \{0,1,2,3,4,5,6\}$
- $\{a,b\} \cup \{a,b,c,d,e\} = \{a,b,c,d,e\}$
- $\{00,01,\} \cup \{01,11,\} = \{00,01,11\}$

Operação de intersecção

A **intersecção** é uma operação que **denota** a **coleção** de **todos os elementos comuns entre dois conjuntos distintos**. A operação de intersecção é denotada pelo símbolo ∩. Assim:

$$A \cap B = \{x \mid x \in A \text{ e } x \in B\}$$

Exemplos:

- $\{a,b,c\} \cap \{c,d\} = \{c\}$
- $\{a,b,c,d\} \cap \{c,d\} = \{c,d\}$
- $\{a,b\} \cap \{c,d\} = \varnothing$

Dois conjuntos A e B são definidos **disjuntos** se $A \cap B = \varnothing$. Assim, os conjuntos {a,b} e {c,d} são disjuntos, pois $\{a,b\} \cap \{c,d\} = \varnothing$.

Operação de diferença

A operação de **diferença entre dois conjuntos A e B** (nesta ordem) define o **conjunto formado por todos os elementos de A que não pertencem ao conjunto B**. Assim:

$$A - B = \{x \mid x \in A \text{ e } x \notin B\}$$

Exemplos:

- $\{a,b,c\} - \{c,d\} = \{a,b\}$
- $\{a,b\} - \{a,b,c\} = \varnothing$
- $\{a,b,c\} \cap \{d,e\} = \{a,b,c\}$

Operação de complementação

A **complementação** é uma operação que define a complementação de um **conjunto A em relação ao conjunto B** como sendo o **conjunto** de **todos** os **elementos de B que não pertencem a A**. Assim, esse conjunto é definido como:

$$\overline{A}_B = B - A$$

Às vezes, esta operação é definida para conjuntos A em relação a outro conjunto B subentendido e, neste caso, escreve-se simplesmente:

$$\overline{A} = B - A$$

Neste caso, diz-se que o conjunto **subtendido** é o conjunto **universo** da **operação**. O resultado da operação é conhecido simplesmente como **complemento** de A.

Produto cartesiano

Outra operação muito realizada em conjuntos é o **produto cartesiano**. O produto cartesiano de dois conjuntos pode ser visto como o **conjunto formado por todos os pares ordenados (a,b)** em que *a* é um elemento de *A* e *b* é um elemento de *B*. Assim:

$$\boldsymbol{A \times B} = \{(\boldsymbol{a}, \boldsymbol{b}) \mid \boldsymbol{a} \in \boldsymbol{A} \text{ e } \boldsymbol{b} \in \boldsymbol{B}\}$$

Um **par ordenado** é visto como uma representação de **dois elementos** que são **separados** por **vírgula** e **delimitados** por **parênteses**. Essa **representação implica** uma **relação** de **ordem** em que o **elemento *a* é anterior ao elemento *b***. Assim, se $a \neq b$, então $(a, b) \neq (b, a)$.

Se A e B são conjuntos finitos, então $|A \times B| = |A| * |B|$. Dessa forma:

$$A_1 \times A_2 \times \ldots \times A_n = \{(a_1, a_2, \ldots, a_n) \mid a_i \in A_i \text{ para } 1 \leq i \leq n\}$$

Exemplos:

Considere $A = \{a,b,c\}$ e $B = \{0,1\}$.

Então A x B =

$$\{(a,0)\ (a,1),\ (b,0),\ (b,1),\ (c,0),\ (c,1)\}$$

e

$$|A \times B| = |A|*|B| = 3 * 2 = 6$$

Conceitos Fundamentais de Linguagem Formal

Linguagem formal pode ser vista como **mecanismos formais** para **representar e especificar linguagens**. Para entender com bastante clareza o que é linguagem formal, primeiro é necessário entender o que é uma linguagem e todos os conceitos e elementos por trás de uma linguagem. Assim, nesta seção, estudaremos todos os conceitos fundamentais de linguagem, começando pelo conceito de símbolos.

Símbolos

Um **símbolo** é uma **entidade abstrata básica sem definição formal**. Um símbolo é uma representação gráfica única e indivisível (RAMOS, Marcus Vinicios Midena; VEGA, Italo Santiago; JOSE NETO, Joao, 2009). **Símbolos** também são **chamados** de **caracteres**, **elementos** ou **átomos** (RAMOS, Marcus Vinicios Midena; VEGA, Italo Santiago; JOSE NETO, Joao, 2009). Vale a ressalva de que cada símbolo é considerado como uma unidade atômica, não importando a sua particular representação visual. São exemplos de símbolos as letras, os dígitos etc., e estes podem ser ordenados. Por exemplo, tomando as letras do alfabeto português, têm-se uma ordenação A < B < C < D < E ... Z. A principal utilidade dos símbolos está na possibilidade de usá-los como elementos em definições de linguagens. Um conjunto de símbolo forma o alfabeto da linguagem.

Alfabeto

Um **alfabeto** é definido como um **conjunto finito** de **símbolos**. Dessa forma:

- um **conjunto infinito não** é um alfabeto;
- o **conjunto vazio** é um alfabeto.

Exemplos de alfabetos:

» {a, b, c}

» {a, e, i, o, u}

» {0, 1}

» ∅

É importante observar que um **conjunto vazio** também pode ser **considerado** um **alfabeto**.

Agora, vamos considerar uma linguagem de programação, JAVA, como exemplo. Uma linguagem de programação é formada por um conjunto de todos os símbolos usados na construção de programas, isso inclui:

- letras;
- dígitos;
- caracteres especiais tais como “<”, “>”, “/”, “|”, entre outros;
- espaços em brancos.

Nesse texto, o **alfabeto** será representado pelo **símbolo** Σ, como, por exemplo, alfabeto Σ = {a, b, c}. Nesse exemplo, o alfabeto Σ é composto pelos símbolos a, b, e c.

Palavra

Uma **palavra** é uma **sequência finita de símbolos** (do alfabeto). Uma palavra também pode ser chamada de **cadeia *de* caracteres** ou **sentença**. Uma palavra sempre está atrelada a um alfabeto. Por exemplo, considerando o alfabeto português, sejam ***C***, ***A***, ***R***, ***R*** e ***O*** símbolos do alfabeto, então ***CARRO*** é uma palavra. Uma **palavra formada unicamente por um símbolo** recebe o nome de **palavra elementar** ou **palavra unitária** (PAPADIMITRIOU, Christos H.; LEWIS, Harry R, 2000). Uma **palavra** pode ser **vazia**, constituída por **nenhum símbolo**. Uma **palavra vazia é representada por** ε.

Prefixo, Sufixo e Subpalavra

Dada uma palavra, é possível extrair o seu **Prefixo**, **Sufixo** e **Subpalavra**. **Prefixo** de uma palavra é **qualquer sequência inicial de símbolos da palavra**. Sufixo é qualquer sequência final de símbolos de uma palavra, ou seja, o contrário de prefixo. Já uma **Subpalavra é qualquer sequência de símbolos contíguos** da palavra.

Observe os exemplos de **Prefixo**, **Sufixo** e **Subpalavra**:

- **abcb** é uma palavra do alfabeto **{a,b,c}**.
- A palavra **abcd** possui os **prefixos**:
  - » ε, **a**, **ab**, **abc**, **abcb**
- Os **Sufixos** de **abcb** são:
  - » ε, **b**, **cb**, **bcb**, **abcb**
- É importante observar que qualquer **Prefixo** ou **Sufixo** de uma **palavra** é uma **Subpalavra**. Dessa forma, os **Prefixos** e **Sufixos** de **abcb** são **Subpalavras** de **abcb**.

Concatenação de Palavras

Outro conceito muito importante quando se fala em palavra é a **Concatenação de Palavras**, também chamado **simplesmente** de **Concatenação**.

Concatenação de palavras é uma **operação binária**, a qual é aplicada **sobre um conjunto de palavra** com o **objetivo** de **associar a cada par de palavras uma palavra formada pela junção da primeira com a segunda** (PAPADIMITRIOU, Christos H.; LEWIS, Harry R, 2000).

Palavras são **formadas** por **símbolos**, assim, pode-se dizer que uma **concatenação** é uma **justaposição** dos **símbolos** que **representam** as **palavras componentes**. Uma operação de concatenação deve satisfazer as propriedades a seguir.

Primeiro devemos **supor** que **v**, **w**, **t** são **palavras** quaisquer. Agora, observe as propriedades:

- **Associativa**
  - » v(w t) = (v w) t
- **Elemento Neutro**
  - » ε w = w = w ε

A **concatenação** de palavras é uma **operação associativa**, dessa forma, é **possível omitir os parênteses**. Então, **v(w t)** pode ser definido, simplesmente, por **v w t**.

Agora, vamos observar alguns exemplos de concatenação de palavras.

Vamos supor que o **alfabeto** seja Σ = **{a, b}**. Assim, para as **palavras v = baaaa** e **w = bb**, vale que:

- **v w = baaaabb**
- **v ε = baaaa**

Concatenação Sucessiva de uma Palavra

Outro conceito muito importante é o conceito de **concatenação sucessiva de uma palavra**. Tal conceito é representado na forma de um expoente. Assim:

$\mathbf{w^n}$, onde ***n*** é o **número de concatenações sucessivas** e w é uma palavra qualquer.

É definida indutivamente a partir de operação de **concatenação binária**, como segue:

$$\mathbf{w^0 = \varepsilon}$$

$$\mathbf{w^n = w\ w^{n-1},\ para\ n > 0}$$

Observe os exemplos de concatenação sucessiva a seguir, todavia, vamos continuar supondo que w seja uma palavra qualquer. Assim:

$$\mathbf{w^3 = w\ w\ w}$$

$$\mathbf{w^1 = w}$$

Para uma **palavra** qualquer, temos:

$$\mathbf{a^5 = aaaaa}$$

$$\mathbf{a^n = aaa\ ...\ a}$$

Se Σ representa um alfabeto, então:

- Σ^* representa o conjunto de todas as palavras possíveis sobre Σ
- Σ^+ denota $\Sigma^* - \{\varepsilon\}$

Por exemplo, considere o Σ = **{a}**, então:

» Σ^* = {ε, a, aa, aaa, aaaa, aaaaa, aaaaaa, ... ,n}

» Σ^+ = {a, aa, aaa, aaaa, aaaaa, aaaaaa, ..., n}

Conjunto de Todas as Palavras

Vimos que uma **palavra** é uma **sequência finita de símbolos sobre um alfabeto**, certo? Um **alfabeto** pode **conter *n* sequências finitas de símbolos diferentes**. Dessa forma, podemos concluir que um **alfabeto** pode **conter *n* palavras diferentes**. Em palavras mais simples, um **alfabeto** pode **conter um conjunto de palavras**. Assim:

Seja Σ um alfabeto, o conjunto de todas as palavras de Σ^* é indutivamente definido como:

- **Base de indução**
  - » $\varepsilon \in \Sigma^*$
  - » para qualquer x $\in \Sigma$, vale que x $\in \Sigma^*$
- **Passo de indução**
  - » Se u e v são palavras de Σ^*, então, a concatenação u v é uma palavra de Σ^*
  - » Dessa forma, uma definição mais formal de palavra sobre um alfabeto Σ pode ser dada como qualquer elemento w de Σ^*, assim:

$$w \in \Sigma^*$$

Observe, agora, um exemplo de conjunto de Todas as Palavras.

Se Σ = **{a,b}**, portanto:

- Σ^+ = {a, b, aa, ab, ba, aaa, ... }
- Σ^* = {ε, a, b, aa, ab, bb, aaa, ... }

Tamanho de uma Palavra

O tamanho (comprimento) de uma **palavra x é denotado por |x|** e é dado pela **quantidade de símbolos que compõem x** (SIPSER, M, 2007). Assim, o tamanho da palavra |CARRO| é 5. Para uma palavra vazia, o seu tamanho é definido como 0, $|\varepsilon| = 0$.

Linguagem Formal

Em uma primeira definição, podemos definir **linguagem** como **uma forma de comunicação**. Todavia, essa definição é muito **informal**. Uma definição mais formal e completa de linguagem pode ser dada como um conjunto de regras que são aplicadas a um conjunto de símbolos para formar palavras utilizadas por indivíduos

em uma comunicação. Segundo (RAMOS, Marcus Vinicios Midena; VEGA, Italo Santiago; JOSE NETO, Joao, 2009), uma **linguagem** pode ser considerada um **conjunto**, **finito** ou **infinito**, **de palavras de comprimento finito formadas pela concatenação de elementos de um alfabeto finito e não vazio**. Portanto, uma linguagem formal ou simplesmente Linguagem T sobre um alfabeto Σ é um conjunto de palavras sobre Σ, assim:

$$T \subseteq \Sigma^*$$

Observe os próximos exemplos;

- O conjunto vazio $\varnothing$ e o conjunto formado pela palavra vazia $\{\varepsilon\}$ são linguagens sobre qualquer alfabeto. Assim:

$$\varnothing \neq \{\varepsilon\}$$

- Os conjuntos Σ^* e Σ^+ são linguagem sobre um alfabeto Σ qualquer. Assim:

$$\Sigma^* \neq \Sigma^+$$

- Agora, vamos supor que o alfabeto seja $\Sigma = \{a, b\}$. Assim, o conjunto de palíndromos, ou seja, o conjunto de palavras que possuem a mesma leitura da esquerda para a direita e vice-versa, sobre Σ, é um exemplo de linguagem infinita. Dessa forma, são palavras desta linguagem:

$$\varepsilon, a, aa, bb, aaa, aba, bab, bbb, aaaa, ...$$

A Figura 1 ilustra a relação entre os conceitos de símbolos, alfabeto, palavra e linguagem.

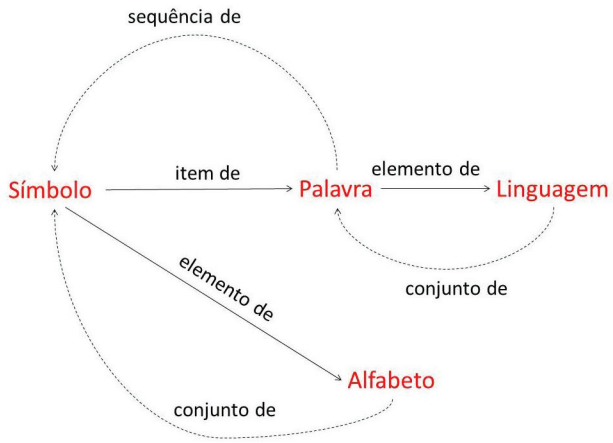


Figura 1 – Relação entre os conceitos de símbolos, alfabeto, palavra e linguagem
Fonte: Acervo do Conteudista

Agora que já conseguimos entender o que é uma **linguagem**, podemos entender com mais clareza o que é uma **linguagem formal**. Uma **linguagem formal** pode ser vista como **mecanismos formais** para **representação e especificação de**

linguagens. Habitualmente, as representações são realizadas por **reconhecedores** e **geradores**. Os **reconhecedores** são mecanismos formais que são utilizados para verificar se uma **palavra pertence ou não pertence a uma linguagem**. Os **Geradores** são mecanismos formais que permitem a **geração de palavras** de uma linguagem. O principal mecanismo de geração é a **gramática**, com destaque para a **gramática de Chomsky** (RAMOS, Marcus Vinicios Midena; VEGA, Italo Santiago; JOSE NETO, Joao, 2009).Gramática e seus elementos são apresentados na próxima subseção.

Gramática

De uma maneira bem sutil, uma **gramática** pode ser definida como um **conjunto finito de regras** que quando aplicadas, **sucessivamente**, **formam palavras**. Na teoria das linguagens formais, o conjunto de todas as palavras geradas por uma gramática denota a linguagem. É interessante observar que as gramáticas usadas para as linguagens naturais, assim como a língua portuguesa, são as mesmas que são usadas pelas linguagens artificiais (linguagem de programação), assim como JAVA. Em determinados momento, as gramáticas podem ser usadas para definir semântica de linguagens.

Gramática de Chomsky

Formalmente, a definição de gramática de **Chomsky** (também chamada de **gramática irrestrita**, ou **simplesmente gramática**) é dada como uma quadrupla ordenada:

$$G = (V,T,P,S)$$

Onde:

- **V** é **um conjunto finito** de **símbolos variáveis** ou **não-terminais**.
- **T** é **um conjunto finito** de **símbolos terminais**.
- **P:** $(V \cup T)^+ \rightarrow (V \cup T)^*$, P é um conjunto finito de pares chamado de **Regras de Produções ou simplesmente Produções**. Cada par da relação é denominado de regra de produção ou simplesmente produção.
- **S** é o **símbolo inicial** ou **variável inicial**.

Na gramática, **uma regra de produção** (α, β) é representada como segue:

$$\alpha \rightarrow \beta$$

Um grupo de regras de produções é representado da seguinte maneira:

$$\alpha \rightarrow \beta_1,\ \alpha \rightarrow \beta_2,\ \alpha \rightarrow \beta_3,\ \alpha \rightarrow \beta_4,\ \ldots\ ,\ \alpha \rightarrow \beta_n,$$

Quando o componente do lado esquerdo é o mesmo, podemos utilizar a seguinte abreviação:

$$\alpha \rightarrow \beta_1\ |\beta_2|\beta_3|\beta_4|\ \ldots\ |\beta_n|$$

Nesta disciplina, por convenção, iremos representar os **símbolos terminais** por **letras maiúsculas**, tais como: **A**, **B**, **C**, **D**, **...** , **Z**. Os **símbolos terminais** serão representados pelas **letras minúsculas ou por dígitos**, assim como **a**, **b**, **c**, **d**, **... z** *e* **1**, **2**, **3**, **4**, **5**, **6**, **...**, **n**.

Derivação

Na gramática, as **regras de produção denotam as condições de geração das palavras da linguagem**, ou seja, definem como as **palavras serão geradas**. A **aplicação** de uma **regra de produção** é chamada de **derivação** de uma **palavra**. Ainda, a aplicação sucessiva de regras de produção permite derivar as palavras da linguagem representada pela gramática.

Formalmente, a relação de derivação é definida como segue:

Seja **G = (V, T, P, S)** uma **gramática**, então, uma derivação é um **par da Relação de Derivação** definida por $\rightarrow$ com domínio em $(V \cup T)+$ e condomínio em $(V \cup T)^*$. Um par de (α, β) da relação de derivação é representado de forma infixada como:

$$\alpha \rightarrow \beta$$

A relação de derivação é definida da seguinte maneira:

- Para toda produção da forma $S \rightarrow \beta$ (S é o símbolo inicial de G), o seguinte par pertence à relação de derivação:

$$S \rightarrow \beta$$

Uma **derivação** é a **substituição** de uma **subpalavra** de acordo com uma regra de produção.

Diversos passos de derivação são definidos da seguinte maneira:

$\rightarrow^*$ **fecho transitivo** *e* **reflexivo da relação**, isto é, zero ou mais passos de derivações sucessivas;

$\rightarrow^+$ **fecho transitivo da relação**, isto é, um ou mais passos de derivações sucessivas.

$\rightarrow^i$ **é realizado exatamente i passos de derivações sucessivas**, com i um número real.

É importante observar que gramáticas são consideradas formalismos de geração porque permitem derivar, isto é, "gerar" todas as palavras da linguagem que representam.

Uma **linguagem gerada** pode ser definida como:

Seja G = (V, T, P, S) uma gramática, então, a linguagem gerada por essa linguagem é denotada por **L(G)** ou **GERA(G)**, é composta por **todas as palavras** formadas por **símbolos terminais** deriváveis a partir do **símbolo inicial S**, isto é:

$$L(G) = \{w \in T^* \mid S \rightarrow^+ w\}$$

A seguir, são apresentados alguns exemplos de Gramática, Derivação e Linguagem Gerada.

Exemplo 1: Vamos analisar uma gramática capaz de gerar qualquer número natural válido em uma linguagem de programação. Dessa forma, a gramática G = (V, T, P, N), onde:

V = {N, D}

T = {0, 1, 2, 3, 4, 5, 6, 7,8,9}

P ={ N → D,

N → DN,

D → 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 }

gera o conjunto dos números naturais. É importante notar que os zeros à esquerda são distinguidos, por exemplo, 123 de 0123. Vamos derivar o número 243.

	Derivação	Regras usadas
1)	N →	N → DN
2)	DN →	D → 2
3)	2N →	N → DN
4)	2DN →	D → 4
5)	24N →	N → D
6)	24D →	D → 3
7)	243	

Vamos analisar a derivação. A ideia central da derivação é a seguinte: sempre que houver um símbolo não terminal, devemos derivá-lo até chegarmos a um símbolo terminal. Todavia, como devemos realizar as derivações? Devemos **utilizar as regras de produções para realizar as derivações**, sempre **partindo** do **símbolo inicial** da **gramática**.

A gramática **G = (V, T, P, N)** define que o símbolo inicial da gramática é **N**. Dessa forma, só é possível começar a derivação a partir das regras de produções que tenham o símbolo N do lado esquerdo.

A gramática G define as regras de produções como sendo **P ={ N → D, N → DN, D → 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 }**. Assim, podemos utilizar essas regras de produções para gerar o número 243. Devido ao fato do símbolo inicial ser o **N**, só podemos começar com as regras de produções que contém o **N** do lado esquerdo. Dessa forma, poderíamos iniciar a derivação com as regras **N → D** ou **N → DN**. Escolhemos a **N → DN** para começar. Assim, utilizando a regra **N → DN**, de **N** chegamos em **DN**. Na derivação, sempre é **substituído** o símbolo do lado esquerdo pelo símbolo do lado direito da regra. Assim, substituímos o **N** por **DN**.

Agora, utilizando a regra **D → 2**, de **DN** chegamos em **2N**. Podemos utilizar a regra **N → DN** e de **2N** para chegar em **2DN**. Utilizando a regra **D → 4**, de **2DN**

chegamos em **24N**. Utilizando a regra **N → D**, de **24N** chegamos em **24D**. Por fim, utilizando a regra **D → 3**, de **24D** chegamos em **243**. Assim, chegamos ao número que queríamos 243.

Neste exemplo, utilizando a seguinte interpretação indutiva, pode ser dada a gramática em questão.

- **Base de Indução:** todo dígito é um número natural.
- **Passo de Indução:** se n é um número natural, então a concatenação de n com qualquer dígito também é um número natural.

Exemplo 2: Como segundo exemplo, vamos considerar uma gramática que gera palavra duplicada.

Observe a gramática: **G = (V, T, P, S)**, onde:

V = {S, X, Y, A, B, F}

T = {a,b}

P ={S → XY,

X → XaA | XbB |F

Aa → aA, Ab → bA, AY → Ya,

Ba → aB, Bb → bB, BY → Yb,

Fa → aF, Fb → bF, FY → ε}

Agora, vamos derivar a palavra **baba**.

	Derivação	Regras usadas
1)	S →	S → XY
2)	XY →	X → XaA
3)	XaAY →	AY → Ya
4)	XaYa →	X → XbB
5)	XbBaYa →	Ba → aB
6)	XbaBYa →	BY → Yb
7)	XbaYba →	X → F
8)	FbaYba →	Fb → bF
9)	bFaYba →	Fa → aF
10)	baFYba →	FY → ε
11)	baba	

Vamos analisar esta derivação. A gramática G denota que G = (V, T, P, S), onde; os símbolos não terminais são V = {S, X, Y, A, B, F}, os símbolos terminais são T = {a,b}, as regras de produções são P ={ S → XY, X → XaA | XbB |F, Aa → aA, Ab → bA, AY → Ya, Ba → aB, Bb → bB, BY → Yb, Fa → aF, Fb → bF, FY → ε} e o símbolo inicial é S.

Começando pelo símbolo inicial **S** e utilizando a regra **S → XY**, de **S** chegamos em **XY**. Utilizando a regra **X → XaA**, de **XY** chegamos em **XaAY**. Agora, utilizando a regra **AY → Ya**, de **XaAY** chegamos em **XaYa**. A partir da utilização da regra **X → XbB**, de **XaYa** chegamos em **XbBaYa**. Utilizando a regra **Ba → aB**, de **XbBaYa** chegamos em **XbaBYa**. Continuando a derivação, agora, utilizando a regra **BY → Yb**, de **XbaBYa** chegamos em **XbaYba**. Com a utilizando da regra **X → F**, de **XbaYba** chegamos em **FbaYba**. Utilizando a regra **Fb → bF**, de **FbFaYba** chegamos em **bFaYba**. Agora, utilizando a regra **Fa → aF**, de **bFaYba** chegamos em **baFYba**. Por fim, utilizando a regra **FY → ε**, de **baFYba** chegamos em **baba**.

É importante observar que o ε é um elemento **neutro**, dessa forma, **baεba** é igual a **baba**.

Gramática Equivalente

Uma gramática G_1 pode ser equivalente à outra gramática G_2. As gramáticas G_1 e G_2 são ditas equivalentes se e somente se:

$$GERA(G_1) = GERA(G_2)$$

Conclusões e Resumo da Unidade

A teoria da Linguagem formal visa o estudo das teorias relacionadas com as linguagens naturais. A teoria da linguagem formal estuda modelos matemáticos que visam à especificação e o reconhecimento de linguagens naturais, tais como suas classificações, estruturas e propriedades.

O estudo da teoria das linguagens formais é de suma importância para a computação. Na computação, a teoria das linguagens formais possui um papel fundamental nas linguagens artificiais, tal como nas linguagens de programação. Nas linguagens de programação, a linguagem formal possui papel fundamental nas análises léxica e sintática das linguagens de programação.

Uma principal definição de **linguagem** de programação pode ser dada como "**um conjunto de símbolos**". Um conjunto pode ser visto como uma coleção de símbolos onde não são consideradas ocorrências múltiplas destes, como, também, não há relação de ordem entre estes.

Linguagem pode ser vista como um conjunto de regras que são aplicadas a um conjunto de símbolos para formar palavras que são utilizadas por indivíduos para se comunicar.

Uma linguagem possui, como elementos básicos símbolos, palavras e alfabeto. Um **símbolo** pode ser definido como uma entidade abstrata básica sem definição formal. Símbolos são utilizados como elementos na linguagem. Um **Alfabeto** pode ser definido como um conjunto finito de símbolos. Já uma **palavra** é uma sequência

finita de símbolos (do alfabeto). Uma palavra também pode ser chamada de cadeia de caracteres ou sentença e sempre está atrelada a um alfabeto.

Uma definição de linguagem mais clara pode ser dada como um conjunto, finito ou infinito, de palavras de comprimento finito formadas pela concatenação de elementos de um alfabeto finito e não vazio.

Linguagem formal pode ser vista como **mecanismos formais** para **representação e especificação de linguagens**. Habitualmente, as representações são realizadas por **reconhecedores** e **geradores**. Os **reconhecedores** são mecanismos formais que são utilizados para **verificar** se uma **palavra pertence ou não pertence a uma linguagem**.

Os **Geradores** são mecanismos formais que permitem a **geração de palavras** de uma linguagem. O principal mecanismo de geração é a **gramática**. De uma maneira bem sutil, uma **gramática** pode ser definida como um **conjunto finito de regras** que quando aplicadas, **sucessivamente**, **formam palavras**.

Material Complementar

Indicações para saber mais sobre os assuntos abordados nesta Unidade:

Livros

Introdução à Teoria de Autômatos, Linguagens e Computação

Hopcroft; J. E., Ullman ; J. D. **Introdução à Teoria de Autômatos, Linguagens e Computação**. Campus, 2002.

Theory of Computation

Sipser; M. ***Theory of Computation***, India, 2008.

Introdução à teoria da computação

Sipser; M. **Introdução à teoria da computação**, Cengage, 2005.

Linguagens Formais e Autômatos: 3

Menezes; P. B. **Linguagens Formais e Autômatos:** 3. Bookman, 2010.

Referências

GERSTING, Judith. **Fundamentos matemáticos para a ciência da computação**. 5. ed. Rio de Janeiro: LTC. 2004. 538p

HOPCROFT, J. E.; ULLMAN, J. D.; MOTWANI, R. **Introdução à teoria de autômatos, linguagens e computação**. Rio de Janeiro: Campus, 2001.

HOPCROFT, John E. ***Introduction to automata theory, languages and computation***. Massachusetts: Addison-wesley Publishing Cia., 1979. 418 p.

MENEZES, P. B. **Linguagens formais e autômatos**. 6. ed. São Paulo: Bookman, 2011.

PAPADIMITRIOU, Christos H.; LEWIS, Harry R. **Elementos da Teoria da Computação**. 2. ed. Porto Alegre: Bookman. 2000. 344p.

RAMOS, Marcus Vinicios Midena; VEGA, Italo Santiago; JOSE NETO, Joao. **Linguagens formais:** teoria, modelagem e implementação. Porto Alegre: Bookman, 2009. 656 p.

RAMOS, M. V. M. **Linguagens Formais e Autômatos**, Curso de Engenharia de Computação, Universidade Federal do Vale do São Francisco. 22 de abril de 2008. Disponível em: <http://docs.fct.unesp.br/docentes/dmec/olivete/lfa/arquivos/Apostila.pdf>. Acesso em 27/12/2018.

SIPSER, M. **Introdução à teoria da computação**. 2. ed. São Paulo: Thompson Learning, 2007.

VIEIRA, Newton José. **Introdução aos fundamentos da computação:** linguagens e máquinas. São Paulo: Thomson, 2006. 319 p. ISBN 8522105081.


Cruzeiro do Sul
Educacional